

## Research Topics Report

### 1. What does ApexaiQ do? What industry problem does it solve?

>ApexaiQ is a SaaS-based, agentless platform that provides continuous IT asset management, visibility, and cyber-attack surface management. It aggregates device, hardware, software, and network data into a single dashboard, delivering a quantified risk score to help organizations track and remediate technical vulnerabilities. It translates complex tech vulnerabilities into a single numerical score (The Apexa iQ Score) ranging from 60 (poor) to 160 (genius), operating much like a credit score for IT health. It is a product based company as their core offering is a cloud-based software subscription platform. Customers pay to log in and use the software independently.

Industry problem it solves:

- ✧ Securing blind spots: In big companies many devices get connected in a system, such as employees smartphones , tv etc. these are unknown and invisible. If a hacker breaches an unmonitored smart TV, they can use it as a backdoor to access the company's entire internal network. ApexaiQ find these hidden devices so they can be secured.
- ✧ Organizing the data: Companies use dozens of different software tools, which don't communicate with each other ApexaiQ pulls data from all these separate tools and cleans it up into single list.
- ✧ Vulnerability Overload: The security tools gather thousands of alert for the company , these cannot be managed by the team at once so apexaiq figures out which are actually dangerous and which can wait and be treated later.

### 2. What is IT asset management and why companies need asset management software?

>IT Asset Management (ITAM) is a business practice that combines financial,

inventory, and contractual tracking to manage an organization's hardware, software, and cloud resources through every stage of their lifecycle. Financial tracking monitors all costs tied to an asset from the day it is purchased until it is thrown away or recycled. It focuses on the money. Inventory tracking maps out the physical location and operational status of every asset. It focuses on physical presence and usage. Contractual tracking manages the legal agreements, promises, and rules paperwork between your company and technology vendors. It focuses on legal compliance. The IT asset management include hardware, software, cloud resources, and lifecycle stages.

Need of asset management software:

Companies need specialized IT Asset Management (ITAM) software because tracking technology manually on spreadsheets quickly becomes impossible. A spreadsheet cannot alert you when a software license expires, scan a network for unmapped security risks, or automatically calculate hardware depreciation. Thus a ITAM software is needed to maintain all the assets.

- Elimination of Waste & Unnecessary Costs: The software identifies employees using different paid tools for the same job. Automatically flags and reclaims software licenses assigned to employees who left the company months ago. It also uses historical data to identify needs of company and avoiding unnecessary purchase for companies.
- Security and Vulnerability reduction: Scans the company network to find unauthorized devices or unapproved software downloaded by employees. Also tracks machines and give security updates to the company. Eg. Tracks which machines are running outdated operating systems and alerts IT to force security updates before hackers exploit them
- Legal and Audit Protection: The software reviews company data to verify that a company's actual technology inventory matches its official records and legal contracts to avoid penalties. It generates reports for the companies without scrambling for the data manually.
- Improves work efficiency: ITAM software automates manual tasks so the IT department can focus on major technical upgrades rather than paperwork. Instead of digging through spreadsheets, they can see data such as new users instantly. Eg. When a new employee is hired, the software automatically builds an "asset kit" based on their job role. The

correct hardware, security access, and software accounts are provisioned without human error.

### **3. 3-5 competitors of ApexaiQ and how they are different from Apexa.**

#### **Case studies**

##### **>Competitors and How They Differ**

- runZero (formerly Rumble)
  - ✧ What they do: Provides agentless cyber asset attack surface management and network visibility.
  - ✧ How it differs: runZero focuses heavily on active and passive scan-less network probing to find unknown industrial (OT) and standard IT devices quickly. ApexaiQ leans more into holistic risk scoring, patch validation workflows, and compliance-driven scoring metrics rather than pure network discovery.
- Vicarius
  - ✧ What they do: Offers a vulnerability remediation platform.
  - ✧ How it differs: While ApexaiQ discovers and scores risk across broad assets, Vicarius specializes in the actual execution layer—providing real-time patching and patchless protection mechanisms. ApexaiQ prioritizes what to fix, whereas Vicarius focuses directly on automated tactical patching.
- CloudWize
  - ✧ What they do: Provides cloud security posture management and compliance automation.
  - ✧ How it differs: CloudWize is natively built for multi-cloud threat monitoring and cloud misconfiguration detection. ApexaiQ delivers a broader hybrid scope spanning classic IT hardware, software catalogs, edge devices (like IGEL endpoints), and traditional environments alongside cloud components.
- Quod Orbis
  - ✧ What they do: Specializes in Continuous Controls Monitoring (CCM).
  - ✧ How it differs: Quod Orbis measures and automates the validation of security controls across the enterprise stack. ApexaiQ centers its foundation on inventory intelligence, asset context, and tracking device compliance/obsolescence over time.

### **4. Why is ApexaiQ an agentless platform?**

ApexaiQ is an agentless platform because it integrates directly with existing security tools, networks, and cloud environments to gather data in near real-time, removing the need to install software agents on individual enterprise devices, servers, or endpoints.

- **Fast and Easy Setup:**
  - ✧ You do not need to install software on thousands of computers. It hooks directly into the network or cloud tools you already use. You start getting data right away without waiting weeks for deployment.
- **Low Maintenance Costs:**
  - IT teams do not have to waste time troubleshooting crashed agents. You only update the main system, never individual computers.
- **Total Visibility**
  - ✧ It spots smart TVs, office printers, and routers that cannot run software.
  - ✧ It detects unauthorized devices employees bring into the workplace.
  - ✧ If a device joins the network, the platform sees it instantly

## **5.Document your findings and research on Cybersecurity.**

>

- **Operating System :**
  - Understood windows and linux os fundamentals.
  - Explored various linux commands such as cd, cat, mkdir etc.
  - Used kal linux and Metasploit framework to understand vulnerabilities and basic ethical hacking concepts.
- **Networking:**
  - Understood network basics such as ip addressing , network devices, network protocols and there port numbers, used nmap tool in kali for network scanning and identifying ports.
- **TryHackme:**
  - Started with basic tryhackme labs such as linux fundamentals , eternal blue where the target machine was hacked by attackers machine.

**Apexaiq score:**

- A numerical risk score that tells a company how healthy their technology is. It uses a scale from 60 to 160.
- It acts as a ranking system, computes security gaps , vulnerabilities etc. and will give the score accordingly
- A high score means the systems are safe, updated, and running smoothly. A low score means the company is at high risk for hacks or system crashes.

### **IT asset management:**

- The IT asset management manages all the assets(devices) in the network.
- This feature acts like an automatic tracker that finds every laptop, server, smartphone, and smart device connected to the company network.
- It eliminates "Shadow IT"—which is when employees buy and use software or gadgets without telling the IT department.

### **Vulnerabilities:**

- A vulnerability is a weakness or an open door that leaves your system exposed to attacks or trouble.
- Hackers use Vulnerability to attack a system as they can enter through it.
- Software bugs, weak authentication, human errors are some types of vulnerability.

### **Obsolescence:**

- Obsolescence occurs when a product becomes old cause a newer, superior technology replaces it.
- The original item still works perfectly, but people stop using it because the new alternative is faster, cheaper, or more efficient. This helps in quicker replacement before the breakdown of old model.
- Eg. DVDs and CDs replaced by digital streaming services

### **Compliance:**

- Compliance is the practice of following all external laws, government regulations, and industry standards, as well as internal company policies.
- It acts as a system of rules and checks to make sure the business operates legally, safely, and ethically.
- Policies ,certifications, audit reports which are verified by the third party are

used to prove compliance of a company.

### **Maintenance:**

- Maintenance is the continuous work required to keep hardware, software, and networks safe, fast, and operational.
- This includes constantly updating software to lock out hackers and fix bugs.
- Also include tracking the physical wear of servers and laptops to replace them before they crash.

### **End of Life, End of Support, End of Maintenance:**

These are the stages or milestones of a product in its lifecycle.

#### ➤ End of Life (EOL):

This is the stage when manufacturer stops the production or officially retires the product.

Sales, marketing, and active commercial distribution stop.

#### ➤ End of Support (EOS):

This is the final milestone where a vendor completely cuts ties with a product.

The vendor stops fixing security vulnerabilities, no technical assistance is given and the product will eventually stop working

#### ➤ End of Maintenance (EOM):

This is the stage where a vendor stops doing routine work on the product, the company will no longer spend time or money to improve it.

No new tools will be there , but you can still call or email customer support for troubleshooting help.

### **Asset Hygiene:**

- Asset hygiene is a practice of tracking, maintaining and cleaning up the company data or assets.
- The data is collected such as information of all devices, the overlapping or repeated record are cleaned up or merged, old data or assets are deleted. Items are labeled by, risk level, function, or owners.
- This stops attackers from using forgotten or unpatched devices as entry points, also stops spending on unused licenses or redundant hardware.

**Crown Jewel:**

- Crown jewel refers to any high-value digital asset, system, or application dependency that is absolutely critical to a company or business's continuity, revenue generation, or regulatory compliance.
- Eg. In bank the core transaction processing software is a crown jewel.  
The core software code is a tech company's crown jewel.

**Inventory:**

- The Inventory is a complete, live list of every single piece of technology the company owns.
- Replaces manual tracking by continuously scanning the network to locate every asset or device or cloud resources.

**NVD (National Vulnerability Database):**

- NVD (National Vulnerability Database) is the U.S. government's official repository of standards-based cybersecurity vulnerability management data.
- It is managed by the National Institute of Standards and Technology (NIST), the database synchronizes with the public Common Vulnerabilities and Exposures (CVE) list. It adds vital contextual analysis to those listings to help IT and security teams automate vulnerability management, security measurement, and compliance.
- The NVD rates security flaws using the Common Vulnerability Scoring System (CVSS)
- Flaws are categorized by their underlying architectural or coding error using the Common Weakness Enumeration (CWE) framework.

**Patch Management:**

- Patch management is the systematic process of identifying, testing, and deploying software updates called patches, which fix security vulnerabilities, resolve bugs, and maintain system stability across an IT infrastructure.
- There are various stages of patch management:
- ✧ The network is scanned and the asset list of all devices are identified.

- ✧ Patches are evaluated based on severity ratings (like CVSS/CVE data), active exploits, and business exposure.
- ✧ Then the patch or update is tested to check whether it cause any disruption.
- ✧ Then, roll out the verified updates in phased stages and successful installation across all endpoints and log compliance records for regulatory standards is confirmed.

### **Data Breaches:**

- A data breach is a security incident where private, sensitive, or confidential data is accessed, copied, stolen, or used without permission. This includes passwords, identity records etc.
- Common Causes of data breaches:
  - ✧ **Phishing:** Fake emails or sites trick people into giving away login codes.
  - ✧ **Weak Passwords:** Hackers guess easy keys or use automated tools to break in.
  - ✧ **Stolen Credentials:** Criminals use login details leaked from older hacks to get into new accounts.
  - ✧ **System Flaws:** Outdated software or misconfigured cloud setups leave doors

### **MSP (Managed Service Provider)**

- A MSP(Managed Service Provider) is an outsourced third-party firm that remotely manages a company's entire information technology (IT) infrastructure and end-user systems.
- The responsibilities of MSP includes:
  - ✧ Monitoring, scaling, and managing cloud environments and on-premise hardware.
  - ✧ Implementing endpoint protection, anti-ransomware patches, identity management, and threat mitigation.
  - ✧ Automating continuous data backups and designing bulletproof disaster recovery plans.
  - ✧ Providing remote troubleshooting, automated patch updates, and end-user helpdesk support.

### **Device Types:**

Device Types refer to the classifications used to group physical hardware or virtual systems based on their function, operating system, network behavior, or how they interact with users.

- ✧ End-user Devices (Endpoints):  
The devices which are used for direct interaction of human with data and applications.  
E.g. Laptops, mobiles, smartwatches etc.
- ✧ Infrastructure & Networking Devices:  
These devices move data and connect other systems in a network. They are the backbone of any network.  
E.g. Routers, switches, Wireless Access Points (WAPs) [ Hardware that allows Wi-Fi-enabled devices to connect to a wired network.]
- ✧ Data Center & Storage Devices:  
High-power systems designed to run enterprise applications and hold large amounts of information.  
E.g. Servers
- ✧ Internet of Things (IoT) & Smart Devices:  
Everyday physical objects embedded with sensors, software, and internet connectivity.  
E.g. Security cameras, smart printers etc.
- ✧ Virtual & Cloud Devices:  
Software-defined equipment that acts exactly like physical hardware but exists entirely in a digital or cloud infrastructure.  
E.g. Virtual Machines, Cloud Instances

### **True Saas:**

- True SaaS (Software as a Service) is software that is built from day one to live completely on the internet.
- Everyone shares the same system: Thousands of different companies log into the exact same application code.
- Automatic updates for everyone: When the software creators add a new button or fix a bug, it changes for all users overnight.

- No software to download: no need to install anything on your computer or office servers, just open a web browser, log in, and start working.

### **Inbound/Outbound Integration:**

- Inbound and outbound integrations describe the direction that data flows between your main system and an external application.
- Inbound integration means your system receives data or requests sent from an outside source  
The third-party or external application makes an API call or sends a file to your system's endpoint. Your system acts as the receiver and server.
- Outbound integration means your system initiates a request to send data out to an external destination.  
Your system triggers an event and makes an API call or file export to an external endpoint.  
Your system acts as the client and sender.

### **Compliance Standards - e.g. CISA, CISO, HIPAA, ISO 27001:**

- Compliance and security standards provide structured rules or frameworks to protect data, lower risk, and build trust.
- CISO
  - ✧ A Chief Information Security Officer (CISO) is a senior, executive-level corporate job title. This individual is responsible for a company's entire data and cybersecurity strategy.
  - ✧ Aligning cybersecurity goals with overarching business goals.
  - ✧ Ensuring the company successfully meets standards like ISO 27001 or laws like HIPAA.
  - ✧ Reporting cyber risks directly to the CEO and Board of Directors.
  - ✧ Overseeing the teams that deploy firewalls, monitor threats, and handle incident responses.
- CISA
  - ✧ The Cybersecurity and Infrastructure Security Agency (CISA) is an official operational component of the United States Department of Homeland

Security (DHS). It is a regulatory and advisory agency, not a standard.

- ✧ Protecting vital public physical and cyber assets from cyberattacks and natural disasters.
- ✧ Issuing public alerts, advisories, and patches for known security vulnerabilities.
- ✧ Acting as the central hub for national cyber defense and coordinating emergency responses to major hacks.

➤ HIPAA

- ✧ The Health Insurance Portability and Accountability Act (HIPAA) is a federal law in the United States designed to protect sensitive patient health data.
- ✧ Restricts how Protected Health Information (PHI) can be used and shared.
- ✧ Mandates specific administrative, physical, and technical safeguards to secure electronic PHI (ePHI).
- ✧ Requires notifying patients, the government, and sometimes the media if a data breach occurs

➤ ISO 27001

- ✧ ISO 27001 is a globally recognized international standard for managing information security. It outlines how to build, implement, maintain, and continually improve an Information Security Management System (ISMS).
- ✧ Finding risks to company data and deciding how to handle them.
- ✧ Ensuring leadership actively funds and supports security rules.
- ✧ A list of 93 security safeguards (in the updated version) covering physical, technological, human resource, and organizational security.

**Perimeter:**

- A perimeter refers to the outer boundary or edge that separates a trusted internal environment (like a corporate network or building) from an untrusted external world (like the public internet or outside areas).
- It serves as the primary line of defense to monitor, filter, and block unauthorized access or threats.
- Inspects incoming and outgoing data packets, blocks malicious traffic, and controls user entry points.
- Components include firewalls, VPNs, routers etc.

## **ROI (Return on Investment), KPI (Key Performance Indicators):**

### ➤ Return on Investment (ROI)

ROI measures the financial profitability or efficiency of a technology investment. It compares the financial gain of a project against its total cost.

- ✧ Calculation formula: Net profit divided by total cost multiplied by 100.
- ✧ Hard ROI: Direct financial impacts like reduced software licensing costs or lower server expenses.
- ✧ Soft ROI: Indirect benefits like hours of manual labor saved or improved customer satisfaction ratings.
- ✧ Tech context: Used to justify cloud migration budgets, automation software purchases, or cybersecurity upgrades.

### ➤ Key Performance Indicators (KPIs)

KPIs are quantifiable metrics used to evaluate the operational success of technology systems, teams, or projects.

- ✧ System KPIs: Uptime percentages, page load speeds, and server response times.
- ✧ Security KPIs: Mean time to detect threats, unpatched vulnerabilities, and phishing click rates.
- ✧ Development KPIs: Deployment frequency, code churn rates, and bug fix turnaround times.
- ✧ Business alignment: Good tech KPIs directly support broader corporate goals like revenue growth

## **Auto-remediation:**

- Auto-remediation is the automated detection, analysis, and resolution of security threats, system misconfigurations, or IT performance issues without human intervention.
- It uses predefined rules, playbooks, and AI to execute instantaneous corrective actions, minimizing system downtime and closing windows of exposure.
- Reduces threat response and fix times from hours or days down to mere seconds or minutes.
- Eliminates human error during manual troubleshooting or patching.

- Frees up IT and security operations teams to focus on strategic initiatives rather than repetitive tasks.
- Common uses: Cloud Security, IT Operations

### **Network protocols:**

Network protocols are official rules and formats that let different computers talk to each other. They make sure data moves safely, in the right order, and without errors across local systems or the global internet.

- ✧ HTTP/HTTPS: Moves web pages from servers to your browser; HTTPS adds a secure, encrypted layer.
- ✧ DNS: Acts as the phone book of the internet. It turns website names (like example.com) into numerical IP addresses.
- ✧ FTP: Moves files between computers on a network.
- ✧ TCP: Guarantees that data packets arrive safely and in the correct order, though it can be a bit slower.
- ✧ UDP: Sends data fast, but does not check if it arrives or stays in order. Great for live video or games.
- ✧ IP: Assigns numbers (addresses) to devices and routes packets across different networks.
- ✧ ICMP: Tests connections and reports errors, such as when you use a ping command.
- ✧ DHCP: Hands out temporary IP addresses automatically to devices joining a network.
- ✧ LS/SSL: Encrypts data to keep private information safe while it travels.
- ✧ SMTP: Sends email messages from a client to a mail server

### **Due-diligence:**

- Due Diligence is the process of thoroughly auditing and investigating an organization's tech stack, security controls, risk exposure, and compliance history prior to business events like mergers, acquisitions, or vendor onboarding.
- It prevents organizations from unknowingly inheriting hidden security vulnerabilities, technical debt, or costly compliance violations.

### **SOAR (Security Orchestration, Automation, and Response):**

- Security Orchestration, Automation, and Response (SOAR) combine separate security tools and workflows into a single automated operational engine.
- By aggregating security alerts, streamlining incident response tasks, and standardizing workflows through playbooks, SOAR enables SOC teams to investigate threats faster and eliminate tedious manual tasks.

### **Role of ITAM in Zero Trust Security Models:**

- ITAM in Zero Trust Security Models plays a vital foundational role by providing the accurate, realtime contextual data needed to continuously verify every access request.
- Because Zero Trust operates on the principle of "never trust, always verify," access decisions rely heavily on ITAM to confirm the requesting user, device identity, patch status, and overall device health before granting entry.

### **Cyber Asset Attack Surface Management (CAASM):**

- Cyber Asset Attack Surface Management (CAASM) is an emerging security technology focused on discovering, consolidating, and continuously visualizing every asset across an organization's hybrid environment through API integrations.
- By connecting data from existing security tools, network scanners, and cloud platforms, CAASM uncovers blind spots, resolves asset management gaps, and provides security teams with a unified view of their exposure profile.

## **PYTHON RESEARCH TOPICS:**

### **Indentation:**

In Python, indentation is spaces or tabs which at the beginning of a line, used to group a block of code. Unlike other languages such as java, c which use {} for grouping code python uses indentation.

Rules for indentation:

- ✧ All statements inside a block must have the same indentation level.
- ✧ Standard indentation in Python is 4 spaces.

- ✧ Mixing tabs and spaces in indentation is not allowed.
- ✧ Improper indentation will cause an Indentation Error

E.g. the indentation is used in loops, if-else.

```
j = 1
while j <= 5:
    print(j)
    j = j + 1
```

```
if age >= 18:
    print("you are an adult.")
else:
    print("you are not an adult.")
```

### **Comments:**

Comments are non-executable lines of code, which are used to tell information about the program or explain logics which makes the code easier for others to understand.

Types of comments in python:

#### ➤ Single-line Comments:

Start with # and continue until the end of the line.

e.g. `sum = a + b # adding two numbers (also an inline comment)`

#### ➤ Multi-line Comments:

Multiple # lines in the code.

e.g. `# This is line 1`

`# This is line 2`

`# This is line 3`

`print("Hello Python")`

#### ➤ Docstrings (Documentation Strings):

Written using triple quotes `"""` or `'''`, look like multi-line comments but serve a special purpose: they explain how functions, classes, or modules work.

e.g. `def greet():`

`"""This function prints a greeting message"""`

`print("Hello, World!")`

**Functions:**

A function is a reusable block of organized code which performs a specific, isolated task. It helps break your program into smaller parts, modular chunks, preventing repetition of code.

Functions are defined using the `def` keyword, followed by the function name, parentheses `()`, and a colon `:`. In some functions `return` statement is used to send a result back to the caller. Once a `return` statement is hit, execution of that function terminates instantly.

```
e.g. def greet_user():  
    print("Hello! Welcome back.")  
  
greet_user() #calling the function
```

Parameters: The placeholders defined in the function header.

Arguments: The actual values you pass into the function when calling it

```
e.g. def describe_pet(animal_type, pet_name):    #parameters(animal_type and  
pet_name)  
  
    print(f"I have a {animal_type} named {pet_name}.")  
  
describe_pet("dog", "Hamish")    #arguments (dog and hamish)
```

**OOPs:**

Object-Oriented Programming (OOP) is a way of writing code by breaking a program down into digital "objects." Instead of writing a long list of instructions, you organize your code to mirror how real-world things look and behave.

OOP uses two main building blocks:

- ❖ **Class:** A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have.

- ❖ Object : An object is an instance of a Class. It represents a specific implementation of the class and holds its own data.

### The 4 Rules (Pillars) of OOP

#### ➤ Inheritance:

- ❖ Inheritance allows a class (child class) to acquire properties and methods of another class (parent class). It supports hierarchical classification and promotes code reuse.
- ❖ A new class can automatically copy all the traits and behaviors of an old class without rewriting the code.

#### ➤ Encapsulation:

- ❖ Encapsulation is the bundling of data (attributes) and methods (functions) within a class, restricting access to some components to control interactions.
- ❖ A class is an example of encapsulation as it groups together member functions, variables and other related data in a single unit.

#### ➤ Polymorphism:

- ❖ Giving the exact same command to different objects, and letting each object execute it in its own unique way.
- ❖ It allows functions or methods with the same name to work differently depending on the type of object they are acting upon

#### ➤ Abstraction:

- ❖ Abstraction hides the internal implementation details while exposing only the necessary functionality.
- ❖ Hiding the messy, complicated background details and showing only the simple tools needed to use the object.

### **File Handling:**

The random-access memory is volatile, and all its contents are lost once a program terminates. In order to persist the data forever, we use files. A file is data stored in a storage device. A python program can talk to the file by reading content from it and writing content to it.

#### Opening a File:

Python uses the `open()` function to open a file.

e.g. `open("this.txt", "r")`

#### Reading a File:

`Read()` is used to read a file.

```
e.g. f = open("this.txt", "r")
      text = f.read()
      print(text)
      f.close()
```

f.read() reads the complete file, whereas f.readline() reads one line at a time.

Writing to a File:

Write() is used to write into a file.

```
e.g. f = open("this.txt", "w")
      f.write("This is nice")
      f.close()
```

File Opening Modes:

r – Read

w – Write

a – Append

+ – Update

rb – Read in binary mode

rt – Read in text mode

with statement:

The with statement automatically closes the file after use.

e.g. with open("this.txt", "r") as f:

```
      text = f.read()
      print(text)
```

### **Exception handling:**

Exception handling isolates and manages runtime errors to prevent a program from crashing unexpectedly.

e.g.

try:

# Code which might throw exception

```
    print(10 / 0)
```

except Exception as e:

```
    print(e)
```

➤ Try Block: The try block contains the code that might trigger a runtime

error.

- Except Block: The except block catches specific exceptions (e.g., ValueError, KeyError) and executes recovery steps.
- Else Clause: The else block runs code *only* if no exceptions were thrown in the try block.
- Finally Block: The finally block executes cleanup tasks invariably, whether an error occurred or not.

Try With Else Clause:

try:

```
result = 10/0
```

except:

```
print("An error occurred!")
```

else:

```
# Executed if try was successful
```

```
print("The code ran successfully without any errors!")
```

Try With Finally Clause:

try:

```
result = 10 / 0
```

except:

```
print("An error occurred!")
```

else:

```
print("The code ran successfully without any errors!")
```

finally:

```
# ALWAYS executed, regardless of success or failure
```

```
print("This cleanup code runs no matter what!")
```

## **Numpy and pandas:**

Both NumPy and Pandas are popular Python libraries used for data analysis and data science.

Numpy(Numerical Python) :

- Is used for is used for numerical and mathematical operations.
- Numpy works with multidimensional arrays , performs fast mathematical calculations, supports matrices and linear algebra, provides functions for statistical operations.

- NumPy allows operations between arrays of different shapes in many cases. It provides following Statistical Functions to calculate:
  - ✧ Mean
  - ✧ Median
  - ✧ Standard deviation
  - ✧ Minimum
  - ✧ Maximum
- NumPy supports matrix operations, determinants, inverses, and solving linear equations. It can generate random numbers and random arrays.

**Pandas:**

- Is used for storing, organizing, and analyzing structured data.
- Pandas use series for one-dimensional data, uses data frames for two-dimensional tabular data, can read CSV ,JSON ,SQL databases, Excel files, helps in cleaning, filtering, and analyzing data.
- Pandas provides two main data structures:
  - ✧ Series: One-dimensional labeled data.
  - ✧ DataFrame: Two-dimensional tabular data with rows and columns.
- It makes it easy to store, organize, and manipulate large amounts of data.
- Pandas helps in cleaning data by handling missing values, removing duplicate values ,replacing incorrect values.
- Multiple datasets can be combined using operations such as merge, join, concatenate.

### **Selenium for Scrapping:**

Web scraping is the process of automatically collecting data from websites using a program or software. Instead of manually copying information from a website, a web scraping program can extract the required data automatically. Selenium is an open-source tool mainly used for automating web browsers. It is widely used for web application testing, browser automation, and sometimes web scraping of pages where content is dynamically loaded with JavaScript.

### **Regex:**

Regex, or Regular Expression, is a sequence of characters used to search,

match, find, or validate patterns in text.

It is commonly used to find specific information from large amounts of text.

Instead of searching for an exact word, Regex allows you to search for abstract concepts like "any email address," "a 10-digit phone number," or "words starting with a capital letter."

➤ Character Classes:

- ✧ . : Matches any single character except a newline.
- ✧ \d : Matches any digit (0-9). Same as [0-9].
- ✧ \D : Matches any non-digit.
- ✧ \w : Matches any alphanumeric character (letters, numbers, underscores).
- ✧ \s : Matches whitespace (spaces, tabs, newlines).
- ✧ [A-Z] : Matches any uppercase letter within the custom bracket range.

➤ Quantifiers (How many times)

- ✧ \* : 0 or more times.
- ✧ + : 1 or more times.
- ✧ ? : 0 or 1 time (optional).
- ✧ {n} : Exactly n times.
- ✧ {n,m} : Between n and m times

e.g. import re

```
text = "Call me at 555-1234 or text office at 555-5678."
```

```
# Pattern: 3 digits, a dash, 4 digits
```

```
pattern = r"\d{3}-\d{4}"
```

```
matches = re.findall(pattern, text)
```

```
print(matches) # Output: ['555-1234', '555-5678']
```

## **Multithreading/Multiprocessing:**

Multithreading:

Multithreading is a technique in which a single program or process is divided into multiple threads to perform multiple tasks concurrently. A thread is a small unit of execution inside a process.

Multiple threads: A single process can contain multiple threads.

Shared memory: Threads share the same memory and resources.

Concurrency: Multiple tasks can make progress during overlapping time periods.

Lightweight: Threads require fewer resources than separate processes.

Useful for I/O tasks: Commonly used for file operations, network requests, and waiting for responses.

e.g.

```
import threading
```

```
def task():  
    print("Task is running")
```

```
thread = threading.Thread(target=task)
```

```
thread.start()
```

```
thread.join()
```

```
print("Program finished")
```

Thread() creates a new thread, start() starts the thread, join() makes the main program wait until the thread finishes.

Multiprocessing:

Multiprocessing is a technique in which a program uses multiple separate processes to perform multiple tasks. A process is an independent program execution with its own memory and resources.

Main Features:

A program can create separate processes for different tasks.

Each process has its own memory space.

Multiple processes can execute at the same time on different CPU cores.

Useful for tasks that require heavy processing, such as:

- Mathematical calculations
- Image processing
- Video processing
- Machine learning

e.g. import multiprocessing

```
def task():  
    print("Task is running")
```

```
process = multiprocessing.Process(target=task)
```

```
process.start()
```

```
process.join()
```

```
print("Main program finished")
```

Process() creates a new process, start() starts the process, join() makes the main program wait until the process finishes.

## **Concurrency vs Parallelism:**

Concurrency:

Concurrency is the ability of a program to handle multiple tasks during the same period of time.

The tasks do not always have to execute at exactly the same moment. Instead, the system can switch between tasks, allowing all of them to make progress.

The system switches between tasks, so both tasks make progress.

The CPU works on Task A, then switches to Task B, and later switches back.

This switching happens quickly, creating the appearance that multiple tasks are running together.

e.g. Multithreading

Multiple threads perform tasks within the same process.

Useful mainly for I/O-bound tasks, such as:

- Downloading files
- Reading files
- Network requests
- Waiting for a server response

Parallelism:

Parallelism is the technique of executing multiple tasks at the same time. It usually uses multiple CPU cores or processors so different tasks can run simultaneously.

A large task can be divided into smaller independent tasks. The subtasks run simultaneously and their results are combined at the end.

Useful for complex and CPU-intensive operations.

Parallelism is commonly used in:

- Machine learning
- Deep learning
- Image processing
- Video processing

- Scientific calculations

e.g. Multiprocessing

## **SDLC:**

The Software Development Life Cycle (SDLC) is a structured, step-by-step framework used by teams to plan, build, test, and deploy high-quality software. It ensures a systematic workflow and helps align software development with business goals and user requirements.

Stage of SDLC:

- Stage 1: Planning & Feasibility Analysis

This stage defines project goals, assess feasibility, and gather detailed requirements from users and stakeholders.

It also determines whether the project is technically, financially, and operationally feasible.

A project plan or feasibility report is generated in this stage.

- Stage 2: Requirement Specification

In this stage, detailed functional and non-functional requirements are documented clearly and approved by stakeholders.

Checking software requirements, validations, documentation comes under this.

- Stage 3: System Design

This stage includes the system architecture, user interfaces, database layouts, and overall technical blueprint.

- ✧ High-Level Design (HLD): Defines system architecture, technology stack, database design, and major modules.
- ✧ Low-Level Design (LLD): Specifies component logic, APIs, data structures, and workflows.

- Stage 4: Development (coding)

Developers build the software based on the approved design

Write the actual code to build frontend interfaces, backend systems, and databases.

- Stage 5: Testing

This stage checks the software for bugs, security flaws, and performance issues before release.

Testing ensures the software meets requirements and is free from defects before release.

Types of Testing include:

- ✧ Unit Testing: Verifies individual components
- ✧ Integration Testing: Ensures modules work together
- ✧ System Testing: Validates the complete system
- ✧ User Acceptance Testing (UAT): Confirms business requirements are met

#### ➤ Stage 6: Deployment

Release the finished application into the live production environment for end-users.

Continuous Integration and Continuous Deployment (CI/CD) pipelines for faster and reliable releases.

#### ➤ Stage 7: Maintenance

This stage provides ongoing support, fix late-discovered bugs, and apply updates over time.

Which includes Bug fixes, performance tuning, updates, feature enhancements.

### **Agile and Scrum:**

Agile:

- Agile is a development method in which requirements and solutions evolve in cross-functional teams through collaborative effort. Its approach aligns project and product development with the customer's requirements and overall company goals. It is a methodology based on iterative development. This process encourages frequent inspection and adaptation which helps in the rapid delivery of high-quality software.
- Benefits of using agile:
  - ✧ Predictable Costs and Schedule
  - ✧ Flexibility and Adaptability
  - ✧ Iterative Delivery
  - ✧ Customer Collaboration
  - ✧ Continuous Improvement
  - ✧ Increased Transparency
- Disadvantages of using agile:
  - Requires Experienced Team

- Emphasis on Documentation
- Lack of Predictability

#### Scrum:

- Scrum is a lightweight, iterative, and incremental framework. Scrum breaks down the development phases into stages or cycles called "sprints". The development time for each sprint is maximized and dedicated thereby managing only one sprint at a time. The Scrum team has a scrum master and product owner with constant communication daily.
- There are 5 scrum values:
  - ✧ Commitment
  - ✧ Focus
  - ✧ Openness
  - ✧ Courage
  - ✧ Respect
- Benefits of the scrum methodology:
  - ✧ Iterative Progress
  - ✧ Flexibility in Requirements
  - ✧ Enhanced Team Collaboration
  - ✧ Clear Roles and Responsibilities
- Disadvantages of the scrum methodology:
  - ✧ Dependency on Team Communication
  - ✧ Time-Boxed Sprints
  - ✧ Lack of Predictability in Scope
  - ✧ High Demand for Resources

#### Code version control:

- Code version control is a system that tracks, manages, and safeguards changes to source code over time.
- Tracks and records changes to the codebase, maintaining a structured project history.
- Allows multiple developers to collaborate on the same project without overwriting each other's work.
- Developers access and update a central repository, where project files are stored.
- Enables teams to share updates and revert to earlier versions of the project when needed.

### Components of Version Control Systems:

- ✧ Repository: A central location that stores all project files along with their complete change history and metadata like author and commit message.
- ✧ Revision: A specific saved version of a file or project, identified using a unique ID such as a hash or number.
- ✧ Branch: A separate copy of the codebase used to develop features or fix bugs without affecting the main code.
- ✧ Merging: The process of combining changes from one branch into another, which may sometimes require resolving conflicts.
- ✧ Commit: A snapshot of changes made to the codebase at a specific time, used to track and manage project history.

### Types of Version Control Systems:

#### ➤ Local Version Control Systems (Local VCS)

A Local Version Control System (LVCS) stores all project versions on a single computer and is mainly used by one user without remote collaboration.

#### ➤ Centralized Version Control Systems

In a Centralized Version Control System, all the files and their version history are stored in a single central server. Developers connect to this server to access or modify files.

#### ➤ Distributed Version Control Systems

A Distributed Version Control System allows each developer to have a local repository along with a working copy of the project. Changes made in the local repository are not automatically visible to others.

e.g. of cvc: git, cvs, Mercurial, Subversion (SVN) etc.

### **.Documentation (Doc):**

Documentation is written information that explains how software, code, APIs, or a project works. Good documentation helps developers understand, use, maintain, and modify a project. Common examples include README files, API documentation, setup instructions, and code documentation

## **Risk management:**

- Risk Management is a systematic process of recognizing, evaluating, and handling threats or risks that have an effect on the finances, capital and overall operations of an organization.
- It is a sequence of steps that help a software team to understand, analyze, and manage uncertainty.

The risk management process consists of:

- ✧ Risk Identification
- ✧ Risk analysis
- ✧ Risk Planning
- ✧ Risk Monitoring
- ✧

Risks in Software Projects:

- Project Risks:

It concerns various sorts of monetary funds, schedules, personnel, resources, and customer-related issues. A vital project risk is schedule slippage.

- Technical Risks:

It concerns potential style, implementation, interfacing, testing, and maintenance issues. Technical risks conjointly embody ambiguous specifications, incomplete specifications, dynamic specifications, technical uncertainty, and technical degeneration.

- Business Risks:

This type of risk embodies the risks of building a superb product that nobody needs, losing monetary funds or personal commitments, etc.

## **Python coding standards**

### **PEP-8:**

Python coding standards are universally anchored by PEP-8 , the official style guide for writing clean, readable, and professional Python code.

PEP 8 is the official Python style guide that promotes readable and consistent coding practices. Following it ensures that your code is easy to understand and maintain.

- Indentation

Indentation in Python organizes code into blocks and defines the program's structure. PEP 8 recommends using 4 spaces per level to maintain readability

and consistency.

➤ Docstrings

Use docstrings to describe functions or modules. Single-line descriptions use `"""Description"""`, while multi-line explanations use triple quotes spanning several lines.

➤ Line Length

Keep lines within 79 characters to enhance readability. Use parentheses, brackets, or braces to wrap long lines instead of backslashes.

➤ Comments

Use comments to explain code for better readability. Inline comments follow the code after `'#'`, while block comments are placed above the code they describe

➤ Trailing Commas

Use trailing commas in data structures like tuples, lists, or dictionaries to improve readability and simplify adding new elements.

➤ Encodings

Use standard encodings like UTF-8 or ASCII for your Python files to ensure compatibility and avoid issues with international text.

➤ Spaces Around Operators

Use spaces around operators and after commas for readability, but avoid spaces immediately inside parentheses, brackets, or braces.

➤ Naming Conventions

Follow consistent naming conventions: use CamelCase for classes, `lower_case_with_underscores` for functions and variables, and all uppercase with underscores for constants.

## **Comments/Docstring:**

Comments:

Comments are created using the hash character (`#`). Everything after the `#` on that line is ignored by the Python interpreter. It improves code readability.

Types of comments:

➤ **Block Comments:**

Used to explain a complex section of code that follows.

Inline Comments: Placed on the same line as a statement, separated by at least two spaces.

e.g. # Calculate the final price after applying a standard discount

```
base_price = 100
```

```
discount = 0.15
```

```
final_price = base_price * (1 - discount) # Apply 15% discount
```

➤ **Docstring:**

Docstrings are string literals written using triple quotes (""" or "). They are specifically placed immediately underneath the definition of a module, class, method, or function.

Types of docstrings:

➤ **One-Line Docstring**

Used for obvious, straightforward behaviors. Keep the closing quotes on the same line.

```
python
```

```
def square(n):
```

```
    """Return the square of a given number."""
```

```
    return n * n
```

➤ **Multi-Line Docstring**

Used to document complex functions, detailing parameters, return values, and potential side effects. It features a summary line, a blank line, and then a structured description.

e.g. def divide(a: float, b: float) -> float:

```
    """Divide two numbers and return a float.
```

```
    Args:
```

```
        a (float): The numerator.
```

```
        b (float): The denominator.
```

```
    Returns:
```

```
        float: The result of the division"""
```

```
    return a / b
```

**Error handling and logging:**

Error handling:

- Error handling is the process of detecting and managing errors that occur while a program is running so the program does not crash unexpectedly.
- In Python, errors are commonly handled using:
  - ✧ try
  - ✧ except
  - ✧ else
  - ✧ finally

e.g.

try:

```
num = int(input("Enter a number: "))
result = 10 / num
print(result)
```

except ValueError:

```
print("Please enter a valid number.")
```

except ZeroDivisionError:

```
print("Number cannot be zero.")
```

finally:

```
print("Program finished.")
```

- ✧ try: Contains code that may produce an error.
- ✧ except: Handles a specific error.
- ✧ else: Runs when no error occurs.
- ✧ finally: Runs whether an error occurs or not.

Logging

- Logging means recording information about the execution of a program. It is useful for monitoring and debugging applications.
- Python provides the built-in logging module.

e.g

```
import logging
```

```
logging.basicConfig(
```

```
    filename="app.log",
```

```
    level=logging.INFO,
```

```
    format="%%(asctime)s - %%(levelname)s - %%(message)s"
```

```
)  
logging.info("Program started")  
logging.warning("This is a warning")  
logging.error("An error occurred")  
This information will be stored in the app.log file.
```

#### Logging Levels:

- ✧ DEBUG : Detailed diagnostic info used during development.
- ✧ INFO : Confirmation that things are running normally.
- ✧ WARNING : An indication that something unexpected happened, but the app is still working.
- ✧ ERROR: A serious problem; a specific feature failed.
- ✧ CRITICAL : Severe failure; the entire program might stop working.

#### Efficient code:

Efficient code is code that performs a task using less time, memory, and system resources while still producing the correct output.

- Efficient code performs tasks quickly and accurately.
- It uses minimum execution time.
- It also uses memory efficiently.
- Reduces unnecessary use of CPU and other resources.
- It avoids repeated calculations and duplicate code.
- Uses suitable algorithms and data structures.
- Avoids unnecessary nested loops.
- Improves program performance.

#### Various Principles:

- The Zen of Python (Language Philosophy)

Type import this into any Python interpreter to read its core guiding principles written by Tim Peters. These principles dictate how Python itself was built and how "Pythonic" code should look:

- ✧ Code should be clean and aesthetically pleasing.
- ✧ Do not hide what your code is doing; let variables and actions be obvious.
- ✧ Choose the simplest path to solve a problem.
- ✧ Python relies heavily on whitespace and English-like syntax because code

is read much more often than it is written.

- ✧ Unless explicitly caught, always log or raise errors so you know exactly what went wrong.
- ✧ Avoid adding multiple syntaxes that accomplish the exact same task

### ➤ Clean Code Principles

These are foundational software engineering practices applied directly to Python to keep codebases readable and maintainable.

- ✧ PEP 8 Style Guide: The official style blueprint for Python code. It dictates rules like using 4 spaces per indentation level, snake\_case for variables/functions, and PascalCase for classes.
- ✧ Avoid duplicating code. If you are copy-pasting code, extract it into a reusable function, class, or loop.
- ✧ Python heavily favors EAFP (*Easier to Ask for Forgiveness than Permission*) over LBYL (*Look Before You Leap*).
- ✧ Checking if a file exists before opening it.

Trying to open the file directly inside a try/except block and handling the error if it fail

### **Unit testing – validation:**

Unit Testing is a software testing method in which individual units or components of a software application (such as functions, methods, or classes) are tested in isolation to verify that they work correctly as expected.

- It helps find and fix defects at the very beginning of the development cycle, reducing the cost and effort of debugging later.
- It promotes writing modular, clean, and maintainable code by ensuring each small part of the application behaves correctly on its own.

The unit testing workflow includes the following step by step process:

- Identify the Unit to Test: Identify the function, method, class, or module that needs to be tested and understand its expected behavior.
- Create Unit Test Cases: Write unit test cases covering positive, negative, boundary, and exception scenarios for the selected unit.
- Execute Unit Tests: Run the unit tests using a testing framework to verify that the code behaves as expected.
- Analyze Test Results: Review the test results to identify failed test cases and determine the cause of defects.
- Fix Defects: Correct the issues found during testing and update the code

to resolve identified defects.

- Re-execute Unit Tests: Run the unit tests again after fixes to ensure the defects have been resolved successfully.
- Measure Test Coverage: Evaluate code coverage to verify that critical paths, conditions, and branches have been adequately tested.
- Integrate with CI/CD Pipeline: Automate unit test execution within the build process to ensure continuous code quality and early defect detection.

### **Ruff, black:**

Ruff and Black are both high-performance Python code tools, where Ruff provides an extremely fast, Black-compatible code formatter alongside its powerful linter.

- Black:
  - ✧ Black is a rigid, opinionated Python code formatter with zero configuration options.
  - ✧ Written in Python.
  - ✧ A formatter for Python code.
  - ✧ Black only formats code, whereas Ruff lints, fixes errors, and sorts imports.
- Ruff:
  - ✧ Ruff is an extremely fast Python linter and code formatter written in Rust.
  - ✧ Ruff aims to be a drop-in replacement for Black while also combining linting and import sorting
  - ✧ Ruff executes orders of magnitude faster than Black because it is built in Rust.